國 立 清 華 大 學

National Tsing Hua University

碩 士 論 文

Master's Thesis

基於自我確信評估與問題拆解的自適應知識檢索方法

SEEK: Self-certainty Evaluated Evidence-gated Knowledge-retrieval

系所別： 資訊系統與應用研究所

Dept. / Grad. Inst.: Institute of Information Systems and Applications

學號 Student ID: 113065528

研究生 Author： 簡宏諭 Hung-Yu Chien

指導教授 Advisor： 高宏宇 Hung-Yu Kao

中 華 民 國 一一五 年 七 月

July 2026

Abstract

This thesis proposes SEEK (Self-certainty Evaluated Evidence-gated Knowledge-retrieval), a lightweight adaptive Retrieval-Augmented Generation (RAG) framework for open-domain question answering. Fixed retrieval can distract simple questions and miss intermediate entities needed by multi-hop questions. Adaptive RAG must decide both whether to retrieve and whether decomposition can expose evidence without discarding global context. Existing approaches rely on coarse routing, imperfect uncertainty signals, or costly learned policies.

SEEK couples step-level retrieval gating with late path selection. A compact decomposer receives one LoRA supervised fine-tuning pass on 12,000 examples from MuSiQue, 2WikiMultihopQA, and PopQA. At inference, SEEK retains direct and decomposed candidates. It measures how much token probability falls outside the model's five leading candidates; this confidence score, called Tail Leakage, decides whether each sub-question retrieves evidence. Original-question evidence is then added to the decomposed contexts, and a deterministic, training-free selector—the verifier—chooses the final answer using confidence, decomposition stability, and answer type. Decomposition thus remains reversible and routing inspectable.

SEEK coordinates question decomposition, step-level retrieval, and final path selection with lightweight training. Decomposition exposes intermediate evidence needed for multi-hop reasoning; Tail Leakage distinguishes unreliable answers more effectively than maximum token entropy and provides a controllable accuracy–retrieval trade-off; and the dual-path design preserves the complementary strengths of direct answering and decomposed reasoning. SEEK reaches 41.88 EM and 48.90 F1 on average across four datasets.

Keywords: Retrieval-Augmented Generation, Adaptive Retrieval, Multi-hop Question Answering, Tail Leakage, Verifier Routing, Large Language Models

摘要

本論文提出 SEEK (Self-certainty Evaluated Evidence-gated Knowledge-retrieval)，一個用於開放領域問答的輕量化自適應檢索增強生成 (Retrieval-Augmented Generation, RAG) 框架。固定的檢索策略可能爲簡單問題引入干擾證據，卻又無法替多跳問題找出後續檢索所需的中介實體。因此，自適應 RAG 不只需要判斷是否檢索，也需要判斷問題分解能否揭露缺失的證據，或反而遺失原始問題中的全域資訊。既有方法多採用粗略的問題層級決策、不完全可靠的詞元層級不確定性訊號，或透過軌跡建構與偏好最佳化學習成本較高的逐步決策策略。

SEEK 將自適應決策分成兩個相互關聯的層次：每個分解子問題是否需要檢索，以及最終答案應取自原始問題路徑或分解路徑。系統以 MuSiQue、2WikiMultihopQA 與 PopQA 的一萬兩千筆資料，透過單次 LoRA 監督式微調訓練一個輕量分解器。推論時，SEEK 同時保留兩條作答路徑，並觀察有多少詞元機率落在模型的前五名候選之外；這個稱爲「尾端洩漏」的信心分數用來決定各子問題是否檢索。系統接著將原始問題的檢索證據加入各分解步驟的上下文，最後由一組無需額外訓練的規則，根據信心、分解穩定性與答案類型選出答案；本文將這個規則式選擇器稱爲驗證器。此設計使問題分解成爲可撤回的候選策略，並讓路由決策保持可檢視性。

SEEK 能以輕量訓練協調問題分解、逐步檢索與最終路徑選擇。問題分解有助於找出多跳推理所需的中介證據；尾端洩漏比最大詞元熵更能辨識不可靠答案，並在準確率與檢索成本之間提供可控制的取捨；雙路徑設計則保留直接作答與分解推理的互補性。SEEK 在四個資料集平均達到 41.88 EM 與 48.90 F1。

關鍵字：檢索增強生成、自適應檢索、多跳問答、尾端洩漏、驗證器路由、大型語言模型

Contents

Abstract i

摘要 ii

Contents v

List of Figures vi

List of Tables vii

1 Introduction 1

1.1 Background and Motivation 1

1.2 Research Contributions 2

2 Related Work 4

2.1 Adaptive RAG 4

2.2 Reasoning and Knowledge Boundaries in RAG 8

2.3 Evaluation Datasets 11

3 Methodology 13

3.1 Overview 13

3.2 Problem Formulation 14

3.3 Sub-Question Decomposer 15

3.4 Token-Level Confidence Probe 18

3.5 Decomposition-Path Inference 22

3.6 Training-Free Verifier 23

4 Experiments 25

4.1 Datasets 25

4.2 Retrieval Corpus 25

4.3 Metrics and Baselines . . . 26
4.4 Implementation Details . . . 26
4.5 Main Results . . . 27

5 Results and Analysis **28**
5.1 Retrieval Analysis . . . 28
5.1.1 Answer-Containing Recall . . . 28
5.1.2 Gold and Base Decomposition on MuSiQue . . . 29
5.2 Ablation Studies . . . 30
5.2.1 Retrieval Is Necessary After Decomposition . . . 30
5.2.2 Gate Threshold Sensitivity . . . 32
5.2.3 Path-Aware Title Handling . . . 33
5.3 The Two Answer Paths . . . 34
5.3.1 Verifier Routing Decisions . . . 34
5.3.2 Correctness Overlap Between Answer Paths . . . 35
5.3.3 Final Routing and Oracle Headroom . . . 36
5.4 Tail Leakage as a Confidence Signal . . . 36

6 Discussion **40**
6.1 Efficiency Considerations . . . 40
6.2 Why Direct and Decomposition Paths Must Coexist . . . 40
6.3 What Training the Decomposer Buys . . . 41
6.4 Tail Leakage: Useful Signal, Not Correctness Guarantee . . . 41
6.5 Remaining Headroom and Remaining Problems . . . 42
6.6 Threats to Validity . . . 43

7 Conclusions **44**
7.1 Limitations . . . 44
7.2 Future Work . . . 44

References **46**

Appendices

A Supplementary Material **51**

A.1 Decomposition Failures 51

A.2 Confidence Signals 53

A.3 Confidence Errors 54

List of Figures

3.1 SEEK Pipeline 14
3.2 Decomposer Prompt 16
3.3 Answer Prompt 18
3.4 Rényi Reweighting and Error Detection 19
3.5 Same Entropy, Different Risk 20
3.6 Monotonicity of the Leakage Score 21
3.7 Error Rate by Tail Leakage Quantile 22

5.1 Answer-Containing Retrieval Recall Figure 29
5.2 Retrieval After Decomposition 31
5.3 Step-Gate Accuracy–Retrieval Trade-off 33
5.4 Direct-vs-Decompose Correctness Counts 35
5.5 Oracle Routing Headroom 37

A.1 Tail Leakage Distributions 54

List of Tables

3.1 Decomposition Patterns by Dataset 16

4.1 Implementation Details 26
4.2 Main Results 27

5.1 Answer-Containing Retrieval Recall 28
5.2 MuSiQue Gold and Base Decomposition Recall 30
5.3 Retrieval Necessity After Decomposition 31
5.4 Safety of Skipped Retrievals 32
5.5 Threshold Sensitivity 32
5.6 Path-Aware Title Retrieval 33
5.7 Verifier Routing Decisions 34
5.8 Correctness Overlap Between the Two Answer Paths 35
5.9 Effect of Final Routing 36
5.10 Representative Confidence Signals 38
5.11 Maximum Entropy vs. Tail Leakage 38
5.12 Entropy Components and Tail Leakage 38
5.13 Direct-Path Accuracy by Confidence Coverage 39

A.1 Decomposition Failures 51
A.2 Additional Decomposition Patterns 52
A.3 Confidence Signal Definitions 53
A.4 Tail Leakage by Correctness 54
A.5 Confidently Wrong Answers 55
A.6 Repeated-Sampling Errors 55

Chapter 1 Introduction

1.1 Background and Motivation

Large language models (LLMs) have become strong general-purpose generators, but the knowledge encoded in their parameters is necessarily incomplete and fixed at training time. When a question concerns recent events, long-tail entities, or information outside the training distribution, an LLM may still produce a fluent answer even when the underlying facts are unsupported. This behaviour limits the reliability of LLMs in knowledge-intensive applications [1].

Retrieval-Augmented Generation (RAG) mitigates this problem by retrieving external passages and conditioning generation on the resulting evidence [2]. Because the evidence source can be updated independently of the language model, RAG provides a practical way to extend the model's accessible knowledge without repeated model training. A fixed retrieval strategy, however, is not equally suitable for every question. For a simple question that the model can already answer, irrelevant passages can introduce distractors and additional latency. For a complex multi-hop question, one retrieval query based on the original question may fail to expose the intermediate entity needed to locate the next supporting passage [3], [4].

Adaptive RAG addresses this mismatch by changing the retrieval strategy according to the question or the model's state. Existing approaches include question-level complexity and self-knowledge routers [5], [6], uncertainty-triggered retrieval during generation [7], [8], and learned policies that decide retrieval actions at individual reasoning steps [9], [10]. These approaches demonstrate that retrieval need is query-dependent, but each has a different limitation. A question-level route is decided before the intermediate reasoning states are known; uncertainty is an imperfect proxy for correctness; and a learned step-level policy can require additional trajectory construction, supervision, or preference optimisation.

Multi-hop question answering introduces a second decision that is closely coupled with retrieval: whether to decompose the question. Question decomposition can convert

a compositional question into shorter sub-questions that expose bridge entities and form more precise retrieval queries [11], [12]. Nevertheless, decomposition is not uniformly reliable. An incorrect intermediate answer can misdirect later queries, while a short sub-question can omit comparison targets, entity pairs, or disambiguating conditions from the original question. Forcing every question through a decomposed path may therefore discard useful global context even when the retrieved passages themselves are relevant.

This thesis consequently frames adaptive RAG as a *decomposition-routing problem*. A system should decide at two levels: whether each decomposed sub-question requires external evidence, and whether the final answer should be taken from the original-question path or the decomposed reasoning path. Preserving both paths makes decomposition a candidate strategy rather than an irreversible early commitment. This perspective motivates SEEK (Self-certainty Evaluated Evidence-gated Knowledge-retrieval), a lightweight framework that trains a compact decomposer but keeps retrieval gating and final path verification transparent and training-free.

Accordingly, this thesis investigates whether competitive adaptive RAG can be achieved with lightweight decomposition training and interpretable inference-time routing, without learning a separate retrieval policy. Its objectives are to produce structurally reliable and executable sub-questions, examine both the value and the limits of token-level self-certainty for sub-question retrieval, design a context-preserving dual-path procedure with transparent final verification, and evaluate the resulting framework across single-hop and multi-hop open-domain question answering in terms of accuracy, retrieval behaviour, offline adaptation cost, and failure modes.

1.2 Research Contributions

Our contributions are as follows:

1. **Dual-path decomposition routing.** This thesis treats adaptive RAG as a choice between an original-question answer path and a decomposed answer path, rather than only a retrieve-or-skip decision. SEEK keeps both candidates until a final verification step, so decomposition becomes a reversible strategy instead of an early commitment. The two paths prove complementary: in the final traces, an oracle

choice between them would add a further 2.9 to 6.0 EM, and the training-free verifier recovers part of this headroom.

2. **A low-cost executable decomposer.** The decomposer is adapted in a single supervised LoRA [13] pass on 12,000 examples and emits KG-style `entity » relation` sub-questions with explicit `#N` dependencies that resolve into directly executable retrieval queries. Keeping original-question evidence available at every decomposed step preserves global context that short sub-questions may omit, without changing the decomposer or training another retriever.

3. **A simple confidence gate with a measured accuracy–retrieval trade-off.** Tail Leakage, computed from the probability mass the model keeps within its own top-five token candidates during a single greedy pass, gates sub-question retrieval. It captures a simple pattern: answers are more likely to be wrong when more probability falls outside these leading candidates. The score therefore measures only the total leaked mass, regardless of how probability is divided within the top five or distributed across the tail.

4. **Training-free late verification.** A deterministic verifier reads direct-answer confidence, decomposition stability, and answer-type compatibility, and selects the final path so that every routing decision remains inspectable. The oracle comparison quantifies the remaining selection headroom without attributing candidate-generation gains to the verifier itself.

Chapter 2 Related Work

2.1 Adaptive RAG

Retrieval-Augmented Generation (RAG) [2] augments a parametric language model with a non-parametric retriever, allowing the model to condition its generation on external evidence rather than relying only on parameters learned during pretraining. A typical RAG pipeline can be described as five stages [14]:

1. **Indexing.** The source corpus is segmented into passages, and each passage is stored in a form that supports retrieval, such as sparse lexical indexes or dense vector representations.

2. **Query manipulation.** The input question may be rewritten, expanded, normalised, or decomposed so that the retriever can better match it with relevant evidence.

3. **Retrieval.** The system selects candidate passages from the indexed corpus, commonly using sparse lexical matching, dense vector similarity, or a hybrid retrieval strategy.

4. **Post-retrieval processing.** The initially retrieved passages may be filtered, reranked, compressed, or otherwise refined before they are passed to the generator.

5. **Generation.** The language model conditions on the selected evidence and produces the final answer, ideally grounding its output in the retrieved passages.

This retrieve-then-generate paradigm is now widely used to make language model outputs more factual in question answering, explanation writing, and other knowledge-intensive tasks. Its central advantage is that factual evidence can be updated externally without retraining the language model. For multi-hop question answering, however, the quality of the retrieved evidence depends strongly on the query that is issued at each step: a single original question may not contain the intermediate entity needed to retrieve the next supporting passage.

This fixed retrieval assumption creates two well-documented failure modes:

- **Noise injection on easy queries.** For questions the model already knows (e.g. "Who wrote *Hamlet*?"), retrieved passages introduce distractor entities that can override correct parametric knowledge [3], [15].
- **Insufficient evidence on hard queries.** For multi-hop questions, a single retrieval pass rarely surfaces all bridging entities, leading to incomplete or compositionally wrong answers [4].

These two failure modes motivate adaptive retrieval: deciding *when* retrieval is useful, *how much* evidence to fetch, and what signal should determine the gate.

Rather than assuming a fixed retrieve-then-read pipeline, adaptive RAG changes the retrieval path according to the input question, the model state, or the current reasoning step. Some methods decide before generation, using a complexity router or a self-knowledge estimate. Others trigger retrieval from uncertainty observed during generation. A third group learns retrieval actions as part of the generation or reasoning policy. These distinctions structure the discussion below.

Question-level routing. A first line of work makes the retrieval decision before generation. These systems inspect the input question and route it to a direct-answer path or a retrieval pipeline, often using a trained classifier or a self-knowledge estimate.

Adaptive-RAG [5] routes each incoming query to one of three retrieval strategies according to its predicted complexity. Easy questions are answered without retrieval, medium-complexity questions use a single retrieval step, and complex questions are routed to a multi-step retrieve-and-reason pipeline such as IRCoT [4]. The router is a T5-Large classifier trained with automatically derived labels. Each training question is assigned to the cheapest strategy that still produces the correct answer, so the classifier learns a cost-aware mapping from question complexity to retrieval depth. This design reduces unnecessary retrieval compared with always using a multi-step pipeline, but the route itself remains a coarse question-level decision.

SKR [6] approaches adaptive retrieval from the perspective of self-knowledge. Instead of predicting general question complexity, SKR estimates whether the language model is likely to know the answer from its own parametric memory. If the question is judged as known, the model answers directly; if it is judged as unknown, external retrieval

is invoked. This judgement can be implemented with a trained classifier or with memory-based self-knowledge strategies. In the classifier setting, SKR is a binary retrieval gate: it decides whether to retrieve, but does not distinguish between single-step and multi-step retrieval. Therefore, compared with Adaptive-RAG, SKR gives a more direct estimate of whether retrieval is needed, but it still commits to a question-level decision before reasoning begins.

Uncertainty-triggered retrieval. A second line of work delays the retrieval decision until generation, using uncertainty signals from the model itself. These methods are often training-free, since the retrieval decision is derived from token probabilities, entropy, attention patterns, or internal model states rather than from an additional trained classifier.

FLARE [7] moves the retrieval decision from the question level to the generation level. At each step, it first drafts a tentative next sentence and checks whether the sentence contains low-confidence tokens. If the confidence falls below a threshold, the tentative sentence is used to formulate a retrieval query, and the model regenerates the sentence with the retrieved evidence. This design allows retrieval to occur only when the model appears uncertain during generation. However, token probability is only an indirect proxy for factual uncertainty. A low-probability token may reflect lexical variation rather than missing knowledge, and sentence-level regeneration may not align well with multi-hop reasoning, where the natural decision unit is often a sub-question.

DRAGIN [8] further refines dynamic retrieval by considering both when to retrieve and what to retrieve. Its *Real-time Information Needs Detection* (RIND) module estimates whether the current generation state requires external evidence, while its *Query Formulation based on Self-attention* (QFS) module constructs retrieval queries from salient context tokens instead of relying only on the most recent sentence. Compared with FLARE, DRAGIN provides a more context-aware retrieval trigger and query formulation strategy. Nevertheless, its decision is still made at the token-generation level, which can be mismatched with explicit sub-question reasoning, and it introduces additional overhead from attention-based query construction.

SeaKR [16] estimates retrieval need from the model's internal states rather than from output probabilities alone. It activates retrieval when the model exhibits high self-

aware uncertainty and reranks retrieved passages according to how much they reduce that uncertainty. This makes the retrieval signal more closely tied to the model's internal generation dynamics than token-level probability alone. However, the signal is still an estimate of uncertainty rather than a direct measure of answer correctness. A model may be uncertain for reasons unrelated to missing external knowledge, and retrieval may not necessarily improve the answer even when uncertainty is high.

TARG [17] studies adaptive retrieval as a training-free, single-shot decision. It generates a short no-context draft and computes uncertainty from the prefix logits to decide whether retrieval should be invoked. In addition to entropy, TARG considers a top-1/top-2 logit-margin signal and a small-sample variance signal. Its main strength is that it provides a lightweight retrieval gate without training an auxiliary classifier or modifying the language model. However, TARG makes one gating decision for the whole question, even on multi-hop benchmarks such as MuSiQue. Its behaviour in decomposition-based retrieval settings, where each sub-question may need its own decision, therefore remains less explored.

Across this family, uncertainty provides a useful and interpretable signal for adaptive retrieval, but it remains an imperfect proxy. Low uncertainty does not guarantee a correct answer, and high uncertainty does not always indicate that retrieval will help. SEEK addresses this limitation by using Tail Leakage both as a sub-question retrieval signal and as the confidence component of a late verifier, rather than treating a single uncertainty score as a hard router for the entire question.

Learned retrieval policies. A third line of work makes retrieval control part of the model's learned generation or reasoning behaviour, rather than relying on a front-end router or a hand-set uncertainty threshold.

Self-RAG [9] trains the language model to generate special reflection tokens that control retrieval and evaluate retrieved passages. For example, the model can emit `[Retrieve]` to decide whether external evidence is needed, and then use additional tokens to judge whether a passage is relevant, supportive, or useful. This design makes retrieval part of generation itself, rather than a separate preprocessing step. However, it requires fine-tuning with reflection-token supervision and a custom decoding procedure

that interleaves normal text generation with retrieval-related decisions.

DeepRAG [10] formulates retrieval-augmented reasoning as a step-by-step decision process. At each sub-query, the model chooses between answering from parametric memory and issuing a retrieval call. Its training procedure first uses binary tree search to explore both choices for each sub-query, and then applies preference-based calibration to refine the retrieval decision at each sub-query. This design gives it strong sub-question-level retrieval control, but the cost is substantial. Tree-search rollout becomes expensive as the number of reasoning steps grows, preference pairs must be carefully constructed, and the final retrieval policy is learned implicitly rather than exposed as an interpretable score.

A related line of work studies retrieval cost more directly. **Fast-or-Better** [18] trains two complexity classifiers, one prioritising accuracy and the other prioritising retrieval efficiency, and exposes a user-controlled parameter that interpolates between them, so the accuracy–cost trade-off is no longer fixed at a single operating point. This view reinforces an important point for adaptive RAG: retrieval should be evaluated not only by accuracy, but also by the cost required to obtain that accuracy.

SEEK is closest in spirit to DeepRAG because both methods make retrieval decisions at the sub-question level. The key difference is that SEEK avoids tree search and preference optimisation. It uses a single supervised decomposer and replaces the learned retrieval policy with a training-free verifier based on confidence, decomposition stability, answer type, and whether the direct path and the decomposed path give the same answer.

Overall, SEEK occupies the under-explored region of *late verifier-based routing with interpretable, training-free signals*: only a single SFT pass is needed for the decomposer, and the final route is decided by transparent query-level diagnostics rather than a learned policy.

2.2 Reasoning and Knowledge Boundaries in RAG

A parallel thread focuses on *how* to retrieve and reason once retrieval is deemed necessary, independent of the gate.

The foundational technique is **IRCoT** [4], which interleaves chain-of-thought rea-

soning with retrieval: after each generated CoT sentence, the partial reasoning becomes the next query. IRCoT is the canonical "multi-step" retrieve-and-reason implementation, but its retrieval count grows linearly in the CoT length, regardless of whether intermediate steps are factually difficult. Question decomposition provides a complementary way to structure multi-hop QA. Rather than retrieving after each free-form reasoning sentence, decomposition-based methods explicitly split a complex question into simpler sub-questions that can be answered independently or sequentially. Early systems such as **DecompRC** [11] show that compositional questions can be reduced to single-hop reading-comprehension steps, while **BREAK/QDMR** [12] formalises decomposition as an ordered sequence of natural-language operations required to answer the original question. SEEK inherits this decomposition paradigm, but uses it primarily as an inspectable reasoning path: it preserves the original-question context for decomposed steps, evaluates whether the generated sub-questions are useful, and verifies whether the direct or decomposed answer should be trusted.

More recent work synthesises reasoning data to internalise the retrieve-and-reason loop. **Auto-RAG** [19] autonomously synthesises multi-turn retrieval–reasoning trajectories and fine-tunes on them, so that the model plans its own retrieval steps without hand-crafted prompts. **Search-o1** [20] embeds retrieval inside a large reasoning model in an agentic loop, augmenting o1-style long-chain reasoning with on-demand search and a document-reasoning module that distils retrieved passages before they re-enter the chain. **AirRAG** [21] activates the model's intrinsic reasoning by combining Monte Carlo Tree Search with self-consistency to explore alternative solution paths. These methods substantially strengthen the reasoning side of RAG, but they rely on heavy data synthesis or search-time expansion; SEEK instead keeps the reasoning component a single lightweight SFT decomposer and concentrates its novelty in training-free verification.

Every adaptive method above ultimately rests on a model's ability to recognise its own *knowledge boundary* — the line between what it reliably knows and what it must look up. A dedicated line of work studies this capacity directly, and we group it into three strands: *self-knowledge elicitation*, *uncertainty estimation*, and the *interaction between the knowledge boundary and retrieval*.

Self-knowledge and its calibration. A first strand asks whether a model can introspect on what it knows. Kadavath et al. [22] show that LLMs are, to a useful degree, calibrated: a model can be prompted to predict the probability `P(True)` that its own answer is correct, and these self-judgements correlate with actual accuracy — though calibration degrades on out-of-distribution questions. Yin et al. [23] construct a "SelfAware" benchmark of answerable and unanswerable questions and find that LLMs only partially "know what they don't know": self-knowledge can be elicited but lags well behind human performance. From the supervised side, Kapoor et al. [24] argue that models "must be taught to know what they don't know", demonstrating that lightweight calibration-tuning or probing substantially sharpens a model's estimate of its own uncertainty. Yin et al. [25] formalise the notion of a knowledge boundary itself, proposing to benchmark a model by probing the frontier of its reliable knowledge rather than by aggregate accuracy alone.

Uncertainty estimation as a boundary signal. A second strand asks how to *measure* the boundary at inference time. Beyond retrieval-specific methods, recent work on LLM uncertainty quantification compares several confidence estimators for answer correctness, including token entropy, maximum softmax probability, margin-based scores, verbalised confidence, and response consistency [26]. These comparisons show that the reliability of each signal varies across tasks and models. This motivates testing whether entropy is sufficient in SEEK's QA setting, or whether a signal focused on the probability mass escaping the top candidates provides a more discriminative retrieval gate.

Semantic entropy further shows why token-level uncertainty can be limited for natural-language answers: different surface forms may express the same meaning, so uncertainty over token sequences does not always correspond to uncertainty over answers. Kuhn et al. [27] introduce *semantic entropy*, which clusters multiple sampled answers by meaning before computing entropy, so that paraphrases of one correct answer do not register as uncertainty; Farquhar et al. [28] scale this idea and show that semantic entropy reliably detects confabulations across QA and biography generation. This observation is important for SEEK because exact QA answers often involve aliases, entity variants, and short spans. The same concern shapes SEEK's choice of signal: competition among the top candidates is often alias-level variation, so Tail Leakage counts only the probability

mass that leaks out of the candidate set. We nevertheless treat token-level confidence as a cheap diagnostic signal rather than as a correctness guarantee.

The boundary under retrieval. A third strand studies the boundary specifically in the RAG setting. Ren et al. [3] show that retrieval augmentation not only injects evidence but also *reshapes* the boundary: relevant passages let a model answer questions it could not before, while irrelevant or distracting passages can push a previously-correct answer over the boundary into error — precisely the "noise injection on easy queries" failure mode of §2.1. This makes a per-step boundary estimate, rather than a one-shot query-level one, essential when retrieval and reasoning are interleaved.

These three strands form the conceptual foundation of SEEK. The failure modes of §2.1 — spurious retrieval on easy queries, missing retrieval on hard ones, and confident hallucination under a hard confidence threshold — are all symptoms of a poorly localised knowledge boundary. Where confidence-based methods read a single uncertainty proxy and policy-based methods bury the boundary inside a learned policy, SEEK makes the boundary inspectable through a late verifier: confidence, decomposition stability, and answer-type compatibility jointly decide whether the direct or decomposed path should be trusted.

2.3 Evaluation Datasets

The evaluation of adaptive RAG hinges on datasets that span both single-hop and multi-hop complexity. This thesis uses HotpotQA, 2WikiMultihopQA, MuSiQue, and PopQA to cover both multi-hop reasoning and single-hop factual recall.

HotpotQA. HotpotQA [29] is the canonical multi-hop QA benchmark, containing 113K Wikipedia-based questions requiring 2-hop reasoning. Each example provides (a) the question, (b) the gold answer, (c) two gold-supporting paragraphs mixed with eight distractor paragraphs (in the `distractor` setting), and (d) sentence-level supporting facts. Questions fall into two compositional patterns: *bridge* (the answer to sub-question 1 is the subject of sub-question 2) and *comparison* (two entities are compared along a shared attribute). It is a standard evaluation set for adaptive RAG and multi-hop reasoning

systems.

2WikiMultihopQA. 2WikiMultihopQA [30] constructs 192K multi-hop questions by leveraging Wikidata knowledge-graph paths, ensuring each question requires traversal of an explicit reasoning chain. It includes four question types (comparison, inference, compositional, and bridge-comparison) and provides KG-triple annotations alongside textual evidence. Compared to HotpotQA, 2WikiMultihopQA is constructed so that its questions are hard to answer through reasoning shortcuts, making it a more reliable test of compositional reasoning.

MuSiQue. MuSiQue (**Mu**ltihop questions via **Si**ngle-hop **Que**stions) [31] addresses two pathologies of earlier datasets: (i) shortcut answers via single-hop reasoning, and (ii) weak distractor selection. MuSiQue is built by composing 2–4 single-hop questions into chains using strict connectivity criteria, yielding 25K questions of 2-hop, 3-hop, and 4-hop varieties. Importantly, MuSiQue provides the *decomposed* sub-questions as supervision, which makes it a natural training source for decomposer models such as the one used in SEEK. Its high distractor difficulty makes it the hardest of the four datasets in this thesis.

PopQA. PopQA [15] is a *single-hop* entity-centric QA dataset of 14K questions, each derived from a Wikidata subject–property–object triple. Critically, PopQA tags each entity with its *popularity* (measured by monthly Wikipedia page views), enabling stratified evaluation of factual recall on long-tail entities. Despite being single-hop, PopQA exposes the over-decomposition failure mode of decomposer-based pipelines: trained on multi-hop data, a decomposer may emit two-step plans even when one step suffices, harming both EM and efficiency.

Single-hop open-domain QA. For completeness, prior adaptive-RAG papers also evaluate on Natural Questions (NQ) [32], TriviaQA [33], and SQuAD [34]. These are single-hop datasets whose answers typically exist in a single evidence paragraph, and they primarily test whether an adaptive method can correctly *skip* retrieval rather than orchestrate it. This thesis uses PopQA as the representative single-hop test set because it stresses factual recall and over-decomposition on entity-centric questions.

Chapter 3 Methodology

This chapter presents SEEK (**S**elf-certainty **E**valuated **E**vidence-gated **K**nowledge-retrieval), a lightweight adaptive RAG framework that treats question decomposition as a routing problem. SEEK runs both a direct path and a decomposition path, preserves original-question evidence inside decomposed steps, and uses a deterministic, training-free rule set to select between the two candidate answers. We refer to this final selector as the *verifier*. The design follows the view that adaptive RAG is more than a retrieve-or-skip problem: a question may need to be answered directly, decomposed into sharper retrieval queries, or repaired with original-question context after decomposition.

3.1 Overview

Given a question q and an open-domain retrieval corpus $\mathcal{P}$, SEEK produces a final answer $\hat{a}$ through two candidate-generation paths followed by a verifier. Figure 3.1 shows the overall pipeline:

1. **Direct path.** The answerer directly answers the original question q with BM25 evidence retrieved from $\mathcal{P}$, producing a direct candidate a^{dir} and a token-level confidence score σ^{dir} over the generated answer span.

2. **Decomposition path.** The decomposer $\mathcal{D}_\theta$ maps q to ordered sub-questions $\{s_1, \ldots, s_n\}$. Each sub-question is answered sequentially, with placeholder substitution, confidence probing, conditional retrieval, original-question evidence union, and a final synthesis step that produces a^{dec}.

3. **Training-free verifier.** A deterministic verifier compares a^{dir} and a^{dec} using direct-answer confidence, decomposition stability, and answer-type compatibility. It returns either a^{dir} or a^{dec} as $\hat{a}$.

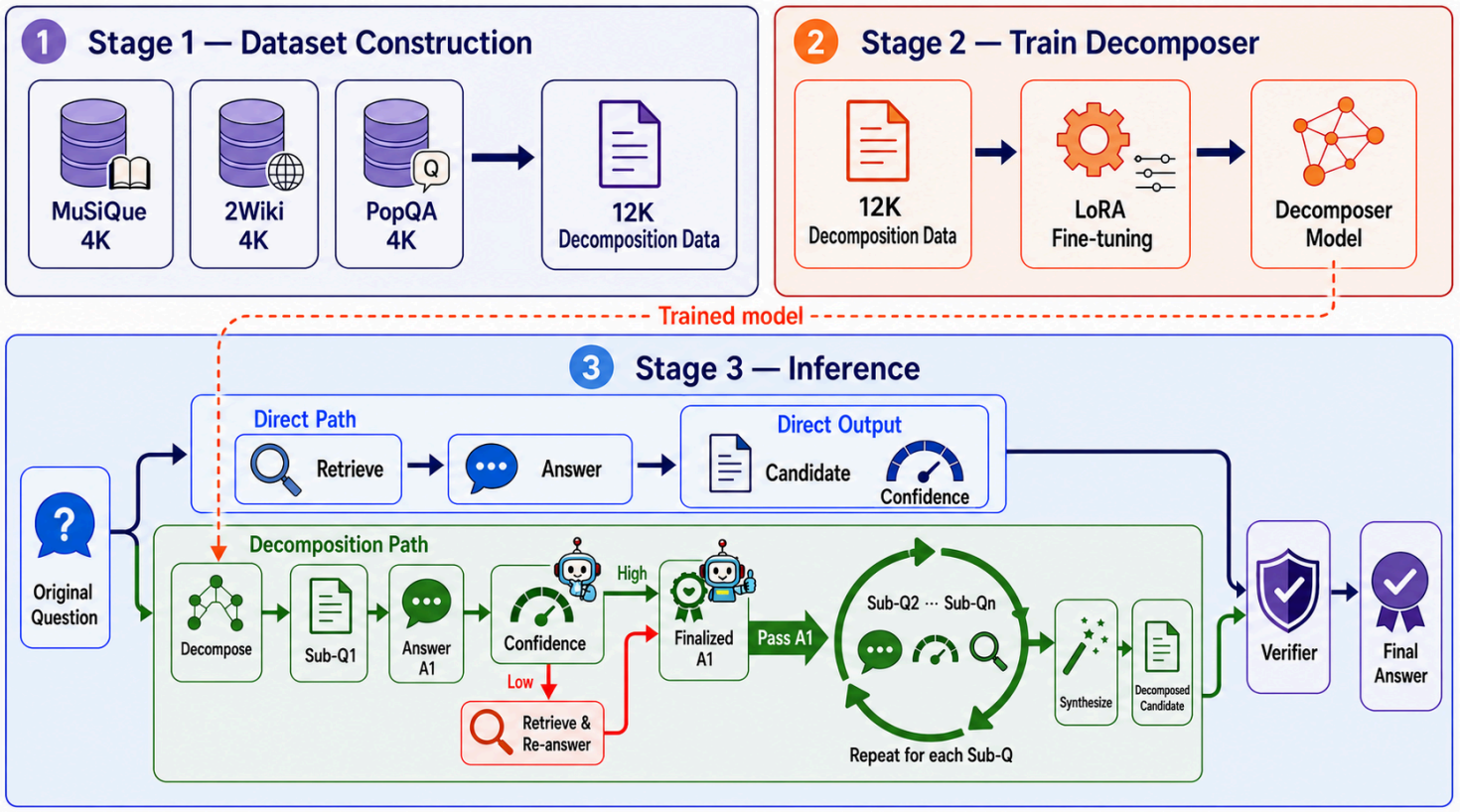


Fig. 3.1: SEEK training and dual-path inference pipeline.

3.2 Problem Formulation

We study open-domain question answering: given a natural-language question q and a passage corpus $\mathcal{P}$, the system outputs a short answer span $\hat{a}$, which is evaluated against gold answers by exact match and F1. The questions of interest are often multi-hop, so a single retrieval with the original question may not surface all the required evidence. Let $\mathcal{M}_\theta$ be a pretrained language model, $r(q, \mathcal{P}, k)$ a BM25 retriever over an open-domain corpus $\mathcal{P}$, and $\mathcal{D}_\theta(q)$ a decomposer that emits sub-questions. A conventional retrieve-then-read system produces one answer from one path. SEEK instead constructs two candidates. The direct candidate answers the original question with retrieved evidence:

$$a^{\text{dir}} = \mathcal{M}_\theta(q, r(q, \mathcal{P}, k)).$$

The decomposition candidate first maps the question to an ordered list of sub-questions,

$$\{s_1, \ldots, s_n\} = \mathcal{D}_\theta(q),$$

then answers each sub-question s_i in order to obtain a_i, and finally synthesises the pairs into an answer span:

$$a^{\text{dec}} = \text{Synth}\left(q, \{(s_i, a_i)\}_{i=1}^{n}\right).$$

Here Synth is not a string-level merge but a final generation pass of the base model $\mathcal{M}_\theta$: it reads q together with all pairs (s_i, a_i) and generates the final answer span under the prompt rules described in Section 3.5.

The final decision is a verifier v:

$$\hat{a} = v\left(q, a^{\text{dir}}, a^{\text{dec}}, z^{\text{dir}}, z^{\text{dec}}\right),$$

where z^{dir} and z^{dec} are inference-time diagnostic signals such as confidence scores, unresolved placeholders, and whether a^{dir} and a^{dec} are the same answer after normalisation. The verifier is a transparent rule set designed to approximate the oracle choice between direct and decomposed reasoning.

3.3 Sub-Question Decomposer

The decomposer emits one sub-question per line, prefixed with `#`$\langle i \rangle$`:`. Each sub-question is expressed as a compact knowledge-graph fragment:

$$\texttt{\#}i\texttt{:} \quad \langle \textit{entity} \rangle \ \texttt{»} \ \langle \textit{relation} \rangle$$

When a sub-question depends on an earlier answer, the placeholder `#`j refers to the answer of step j. For example:

`#1:` `Inception » director` $\rightarrow a_1 =$ `Christopher Nolan`

`#2:` `#1 » place of birth` $\rightarrow$ `Christopher Nolan » place of birth`

At inference, placeholders are resolved sequentially: `#1` stands for the answer text of step 1, not its index. Once step 1 returns a_1, `#1` in s_2 is replaced by that answer

before retrieval and answering, as shown in the example above. The KG-triple notation is compact, consistent across datasets, and aligned with Wikidata-style supervision in 2WikiMultihopQA and PopQA.

The fixed instruction used to prompt the decomposer is shown in Figure 3.2.

Prompt
Decompose the following question into an ordered list of atomic sub-questions. Each sub-question must be answerable in a single retrieval and reasoning step. When a sub-question depends on the answer of an earlier one, refer to that step with a #N placeholder. Output one sub-question per line, prefixed with #<index>:

Fig. 3.2: Fixed instruction prompt used by the decomposer.

The prompt standardises the output format without imposing a fixed decomposition depth. As Table 3.1 illustrates, the same representation accommodates one-hop fact lookup, bridge reasoning, comparison, and longer compositional chains.

Table 3.1: Representative decomposition patterns produced by the SEEK decomposer.

Dataset	Question type	Decomposition
PopQA	1-hop fact lookup	`#1: George Rankin » occupation`
HotpotQA	2-hop bridge	`#1: Film X » director` `#2: #1 » nationality`
2WikiMultihopQA	comparison	`#1: A » date of birth` `#2: B » date of birth`

MuSiQue	compositional chain	`#1: X » capital`
		`#2: #1 » mayor`
		`#3: #2 » party`

Training. Only the decomposer requires training; the answerer, synthesizer, and verifier are training-free. The decomposer is a LoRA [13] adapter on Llama-3-8B-Instruct, trained

on 12,000 examples drawn equally from MuSiQue gold sub-questions, 2WikiMultihopQA evidence-triple conversions, and PopQA one-step triples. Before the PopQA examples were sampled, all 1,000 evaluation question IDs were removed; the resulting 4,000 PopQA training questions have zero exact overlap with the evaluation questions. The PopQA component teaches the decomposer that a one-step plan is valid. This improves single-hop robustness, but Chapter 5 shows that it also creates negative transfer on some multi-hop datasets; the verifier is introduced precisely because no single hard decomposition policy is optimal across all query types. At inference, the adapter is enabled for decomposition and sub-question answering, and disabled for final synthesis, where the base instruction model produces shorter and more stable spans.

3.4 Token-Level Confidence Probe

The fixed answer-only instruction used for both direct and sub-question answering is shown in Figure 3.3.

Prompt
Answer the question with the answer phrase only.
No explanation, no preamble, no full sentence — just the answer.

Fig. 3.3: Answer-only prompt for direct and sub-question generation.

Two kinds of uncertainty. During greedy decoding, SEEK records the softmax distribution $p_t = \text{softmax}(\text{logits}_t)$ for each generated token t. The signal design starts from an observation about how correct and incorrect answers differ in these distributions. When the model knows the answer, the probability mass concentrates on a few candidates, and the remaining competition is typically between surface variants of the same answer, such as aliases, casing, or formatting differences. When the model lacks the required knowledge, the mass instead disperses over many unrelated continuations, well beyond any small set of plausible candidates. What separates the two cases is not how evenly the leading candidates split the mass, but whether the mass stays inside a small candidate set at all.

Locating the signal with Rényi entropy. Before committing to a particular functional form, SEEK first asks *which region* of the next-token distribution carries the error-related information. Rényi entropy [35] offers a controlled instrument for this question, because it leaves the underlying probabilities untouched and changes only the relative emphasis given to high- and low-probability tokens:

$$H_\alpha(p_t) = \frac{1}{1-\alpha} \log \sum_v p_t(v)^\alpha, \qquad \alpha > 0,\ \alpha \neq 1, \tag{3.1}$$

which recovers Shannon entropy in the limit $\alpha \to 1$. For $\alpha > 1$ the measure emphasises the largest probabilities, whereas $\alpha < 1$ makes it more sensitive to the tail. This distinction matters for LLM next-token distributions because Shannon entropy combines

uncertainty among the leading candidates with probability spread across the much larger tail. We therefore replace Shannon entropy with each Rényi variant in the same error detector while leaving the remaining evaluation settings unchanged. Figure 3.4 shows that error detection improves as the weighting shifts from the head towards the tail, indicating that the low-probability region contains more useful error-related information. Rényi entropy still compresses the entire distribution into one number, however, and cannot determine whether the important factor is the total mass in the tail or its internal arrangement. The remainder of this section separates these possibilities.

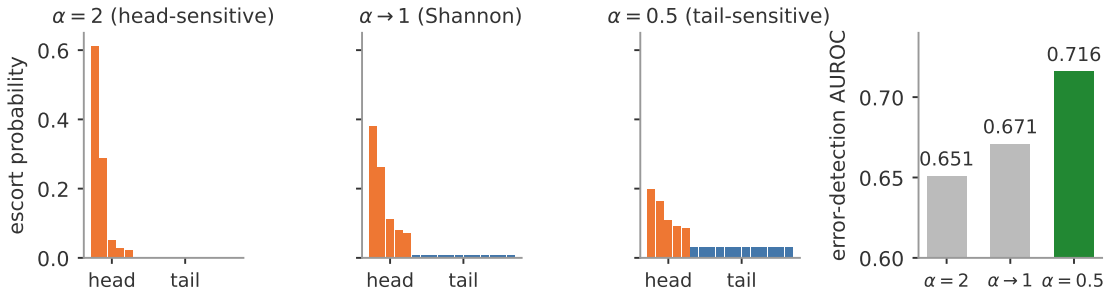


Fig. 3.4: Effect of Rényi reweighting on error detection. Lower α emphasizes the probability tail and improves AUROC on 2WikiMultihopQA.

Same entropy, different risk. Figure 3.5 shows two next-token distributions with the same Shannon entropy and the same leading candidates. Distribution B retains less probability within the top five and therefore places twice as much mass in the tail. Entropy assigns the same value to both distributions, whereas the Tail Leakage score introduced below clearly separates them.

Decomposing token entropy. Token entropy is the usual way to measure this kind of uncertainty, but it mixes the leakage of probability mass out of the candidate set with effects that are unrelated to correctness. Let $V_5(p_t)$ denote the set of the five most probable tokens under p_t, and let

$$m_t = \sum_{v \in V_5(p_t)} p_t(v) \tag{3.2}$$

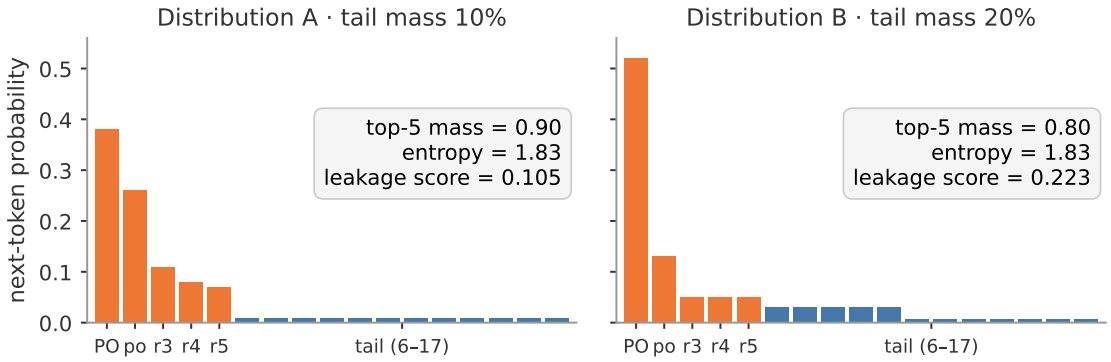


Fig. 3.5: Both distributions have the same Shannon entropy, but Distribution B places more probability mass in the tail.

be the probability mass retained within these candidates; the mass $1 - m_t$ that leaks into the rest of the vocabulary is the tail. For token entropy $H(p_t) = -\sum_v p_t(v) \log p_t(v)$, the chain rule gives three components:

$$\begin{aligned} H(p_t) = &\left[-m_t \log m_t - (1 - m_t) \log(1 - m_t)\right] \\ &+ m_t \, H(\hat{p}_t^{\text{in}}) + (1 - m_t) \, H(\hat{p}_t^{\text{out}}), \end{aligned} \tag{3.3}$$

where the bracketed term is the binary entropy of whether the next token falls inside the top five or in the tail, and $\hat{p}_t^{\text{in}}$, $\hat{p}_t^{\text{out}}$ are the renormalised distributions inside and outside the top-five set. The second term measures competition within the top five, while the third describes how probability is arranged within the tail. Mixing these effects makes entropy less specific as an error signal. This decomposition is an identity rather than a new uncertainty measure; it simply allows the three sources of uncertainty to be evaluated separately. The component analysis in Section 5.4 confirms that the amount of mass leaving the candidate set is more informative than either competition among leading candidates or the arrangement of mass within the tail.

Tail Leakage. The confidence signal of SEEK retains only the top-five mass m_t from this decomposition and discards the two within-group entropy terms. To make the score increase as more probability leaves the candidate set, SEEK takes the negative logarithm of the retained top-five mass and sums the token-level values across the generated answer:

$$\mathrm{TL}_t = -\log m_t, \tag{3.4}$$

$$\sigma^{\mathrm{tl}} = \sum_{t \in \mathcal{T}} \mathrm{TL}_t = -\log \prod_{t \in \mathcal{T}} m_t, \tag{3.5}$$

where $\mathcal{T}$ indexes the generated answer tokens, excluding EOS. $\mathrm{TL}_t = -\log m_t$ is the token-level Tail Leakage score: it is near zero when the top five candidates retain almost all of the mass, and it grows as mass leaks into the tail.

The transform is deliberately *not* the bracketed binary-entropy term of Equation (3.3). A risk score should satisfy a basic monotonicity requirement: the more probability leaves the candidate set, the higher the score, with no reversal at severe leakage. As Figure 3.6 shows, binary entropy violates this requirement, because as a function of the leaked mass $1 - m_t$ it rises, peaks at $1 - m_t = 0.5$, and then falls again, so a token that leaks most of its mass would receive a score as low as one that leaks almost nothing. The role of the decomposition is therefore to identify m_t as the informative variable, and the role of $-\log m_t$ is to convert that variable into a score that is strictly increasing in the leaked mass and, as a logarithm, additive across positions.

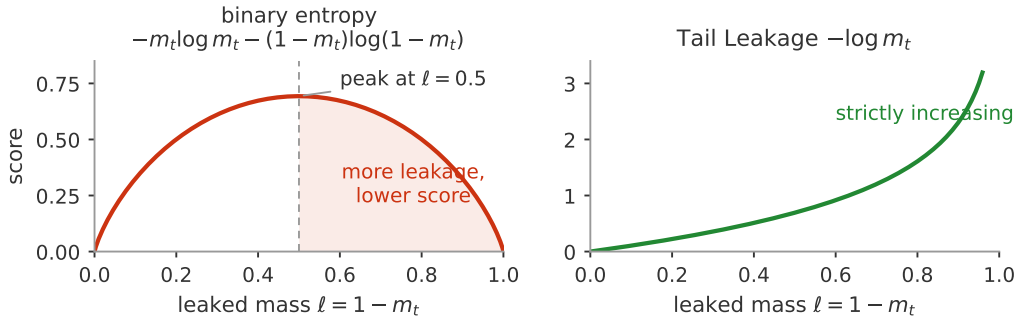


Fig. 3.6: Binary Entropy and Tail Leakage versus leaked probability mass. Only Tail Leakage increases monotonically with leakage.

The answer-level score σ^{tl} accumulates this leakage over the generated span. The sum is preferred over a worst-token maximum because a wrong answer may show small leakage at several positions, which a maximum-based aggregate cannot accumulate. Figure 3.7 shows that answer errors become more frequent as Tail Leakage increases, supporting its

use as a ranking signal rather than a calibrated error probability. Section 5.4 examines its behaviour and limitations in detail. SEEK uses Tail Leakage as the per-step retrieval signal and as both confidence inputs to the verifier.

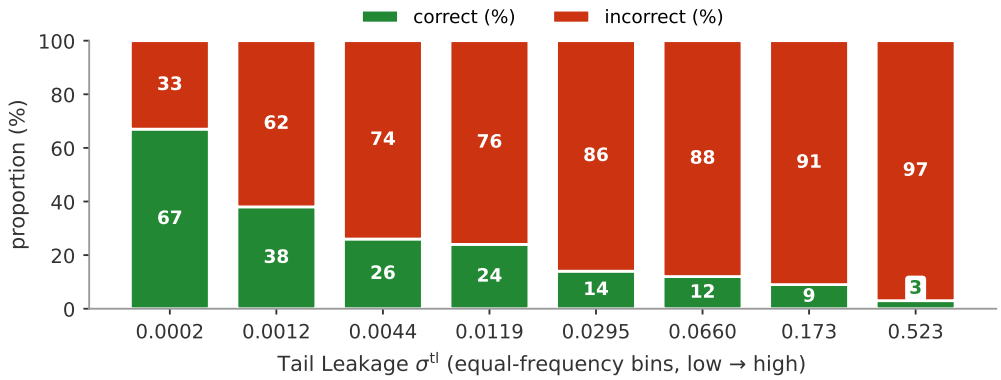


Fig. 3.7: Answer error rate across Tail Leakage quantiles. Error rates increase with Tail Leakage.

3.5 Decomposition-Path Inference

Within the decomposition path, retrieval remains sub-question-level. For a resolved sub-question s_i, the answerer first produces a tentative parametric answer $\tilde{a}_i$ and its Tail Leakage σ_i. The retriever is invoked when:

$$\pi(s_i) = \begin{cases} 1, & \sigma_i \geq \tau \quad \text{(retrieve)}, \\ 0, & \sigma_i < \tau \quad \text{(skip retrieval)}. \end{cases} \tag{3.6}$$

where τ is the retrieval threshold. When retrieval is triggered, BM25 returns top-k passages and the model re-answers s_i using the retrieved context; otherwise $\tilde{a}_i$ is kept.

Path-aware title handling. Original questions and decomposed sub-questions have different query forms. Original questions are longer and contain multiple constraints, whereas sub-questions are usually short and centred on one entity. SEEK therefore disables BM25 title matching for the direct path and enables it for the decomposition path, where the entity name provides a useful title cue. We refer to this path-specific configuration as *path-aware title retrieval*.

Original-question evidence union. A decomposed sub-question is often shorter and more entity-centric than the original question, which improves retrieval precision but can remove global clues such as comparison targets or disambiguating conditions. To preserve this context, the final system augments each retrieved sub-question context with evidence retrieved from the original question:

$$C_i = r(s_i, \mathcal{P}, k) \cup r(q, \mathcal{P}, k). \tag{3.7}$$

The union is deduplicated before answering. This operation does not change the decomposer or train a new retriever; it simply keeps the original-question evidence available when answering each decomposed step. Although these original-question passages are also the evidence source for the direct candidate, the two answer paths remain distinct: the direct path answers the original question directly, whereas the decomposition path still performs sub-question retrieval, step answering, and final synthesis. Thus the union shares evidence, not the direct answer or the final routing decision.

Synthesis. The decomposed answer a^{dec} is produced by a synthesis pass over the original question and all (s_i, a_i) pairs. The prompt encodes three answer-shaping rules derived from error analysis:

1. **Comparison questions** return the matching entity, not the compared value.
2. **Yes-or-no questions** return "Yes" or "No" based on agreement between sub-answers.
3. **Terse-span rule** strips unnecessary modifiers and keeps answer spans short for EM compatibility.

3.6 Training-Free Verifier

The verifier chooses between the direct candidate and the decomposed candidate. It is intentionally conservative: decomposition remains the default because it improves multi-hop retrieval, and the verifier switches to the direct answer only when the direct path is confident and the decomposition path shows signs of instability. The final implementation additionally applies a conservative *type-aware fallback*. The fallback switches to the direct

answer when the decomposed answer does not fit the question, for example a date for a question that asks for a person, or an answer that merely repeats an intermediate step answer; it does not rerun generation and does not train a router.

Signals. The verifier uses three query-level signals:

- **Direct-answer Tail Leakage** σ^{dir}: low Tail Leakage suggests the direct answer may be reliable.
- **Post-decomposition Tail Leakage** σ^{dec}: high Tail Leakage after decomposition and synthesis suggests the decomposed answer is unstable.
- **Structural validity**: unresolved placeholders, empty sub-answers, or malformed sub-questions indicate decomposition failure.

Rule. The verifier chooses direct when the direct answer is confident and decomposition is unstable, and chooses decomposition in the remaining cases. Both conditions are measured with Tail Leakage, using separate thresholds for the direct and decomposed candidates. The type-aware fallback is a deterministic refinement: it only changes the final route when answer type, direct Tail Leakage score, post-decomposition Tail Leakage score, and entity-fragment signals consistently indicate that the direct answer should be preferred.

Chapter 4 Experiments

This chapter specifies the evaluation protocol used to assess SEEK and reports the main comparison with existing methods. It first describes the datasets, retrieval corpus, metrics and baselines, and implementation details, then presents SEEK's headline results under the full-corpus Wikipedia retrieval setting.

4.1 Datasets

We evaluate on HotpotQA, 2WikiMultihopQA, MuSiQue, and PopQA. HotpotQA, 2WikiMultihopQA, and MuSiQue test multi-hop reasoning, while PopQA tests single-hop factual recall and exposes over-decomposition on simple entity-centric questions. All headline results use the first 1,000 questions of each development set without shuffling.

4.2 Retrieval Corpus

The main comparison uses full-corpus open-domain retrieval over a shared Elasticsearch index containing approximately 21 million non-overlapping 100-word passages from English Wikipedia, following the corpus construction of DPR [36]. We use BM25 with top-$k = 8$ for all four datasets. SEEK applies a path-aware title policy: the direct path retrieves with the original question over paragraph text only, while the decomposition path retrieves short sub-question queries over title and paragraph text. This policy is dataset-agnostic because it depends only on query type.

The final system further uses *original-question evidence union*. For each decomposed sub-question, the evidence context is the union of passages retrieved by the sub-question and passages retrieved by the original question. This preserves the global context that can be lost when a long question is decomposed into short sub-questions.

4.3 Metrics and Baselines

We report normalised exact match (EM) and token-level F1. Before scoring, predictions and references are lowercased and stripped of punctuation, English articles, and extra whitespace. For questions with multiple gold aliases, we use the maximum score over all aliases. The main table compares the final SEEK system with representative Llama-3-8B baselines reported by DeepRAG [10]: CoT and CoT-Retrieve as reasoning baselines, FLARE [7] and DRAGIN [8] as training-free adaptive retrieval baselines, and DeepRAG as the strongest learned adaptive baseline. Component-level comparisons are deferred to Chapter 5.

4.4 Implementation Details

All SEEK results use Llama-3-8B-Instruct and the three-source LoRA decomposer described in Chapter 3. Table 4.1 summarises the training and inference configuration. At inference, the adapter is enabled for decomposition and sub-question answering and disabled for synthesis. The final verifier uses Tail Leakage together with the conservative type-aware fallback; it only selects between existing candidates and does not rerun generation or train another model.

Table 4.1: Training and inference configuration used in all SEEK experiments.

Setting	Value
Backbone	Llama-3-8B-Instruct
Decomposer training data	12,000 examples from MuSiQue, 2Wiki, and PopQA
LoRA configuration	rank 16; alpha 32; dropout 0.05; all linear modules
Precision and context length	bf16; 1,024 tokens
Training schedule	3 epochs; effective batch size 16
Learning rate	1×10^{-4}; cosine decay; 10% warmup
Training framework	LLaMA-Factory [37]
Accuracy-first gate	$\tau = 0.002305$

4.5 Main Results

Table 4.2 reports the final results on the four datasets also reported by DeepRAG. The final SEEK system combines path-aware title retrieval, original-question evidence union, Tail Leakage gating for sub-question retrieval, and the type-aware training-free verifier.

Table 4.2: Main results across all datasets.

Type	Method	HotpotQA	2Wiki	MuSiQue	PopQA	Average
Reasoning	CoT	27.20/37.75	28.20/34.85	13.70/22.97	21.20/25.33	22.58/30.23
Reasoning	CoT-Retrieve	34.90/46.85	35.80/43.41	19.10/28.18	32.80/45.87	30.65/41.08
Adaptive	FLARE [7]	23.80/32.88	30.30/37.45	14.50/23.57	28.80/40.61	24.35/33.63
Adaptive	DRAGIN [8]	27.60/38.05	29.10/35.68	11.80/19.97	22.60/28.53	22.78/30.56
Learned adaptive	DeepRAG [10]	**40.70/51.54**	48.10/53.25	22.50/30.40	42.50/47.80	38.45/45.75
Ours	SEEK final	37.80/49.03	**49.30/56.22**	**26.50/34.63**	**53.90/55.73**	**41.88/48.90**

SEEK achieves the strongest average EM and F1 among the compared methods, outperforming the reasoning, training-free adaptive, and learned adaptive baselines in the aggregate.

At the dataset level, SEEK leads on 2Wiki, MuSiQue, and PopQA. As discussed in Section 5.3.1, the PopQA decomposition candidate is a one-step query reformulation rather than a multi-hop reasoning chain.

HotpotQA is the exception, where SEEK remains below DeepRAG. It is therefore the main remaining limitation and motivates the path and routing analyses in Sections 5.3.2–5.3.3.

Chapter 5 Results and Analysis

This chapter builds on the main comparison reported in Chapter 4 and analyses why SEEK works and where it fails. It examines retrieval quality, confidence-based ablations, the complementary direct and decomposition paths, final-trace failure modes, and the limits of Tail Leakage as a confidence signal.

5.1 Retrieval Analysis

This section asks whether decomposition improves retrieval itself, and how much of the improvement depends on the trained decomposer rather than on decomposition as such.

5.1.1 Answer-Containing Recall

Before evaluating final answers, we first ask whether decomposition actually improves retrieval quality. Table 5.1 reports answer-containing Hit@5: a query is counted as successful if at least one of the top five retrieved passages contains the gold answer or one of its aliases. Yes-or-no questions are excluded because the strings "yes" and "no" are not meaningful evidence strings.

Table 5.1: Answer-containing Hit@5 under open-domain Wikipedia retrieval. The direct path searches paragraph text, while decomposition also uses titles.

Dataset	N	Original	Decomposed	Decomp − Original
2Wiki	917	32.61	**60.63**	+28.03
HotpotQA	941	47.40	**52.92**	+5.53
MuSiQue	1000	19.00	**38.80**	+19.80
PopQA	1000	**71.20**	71.10	-0.10

Decomposition improves answer-containing retrieval on all three multi-hop datasets, with the largest gain on 2Wiki. Figure 5.1 visualises this contrast: the single-hop PopQA

result remains nearly unchanged.

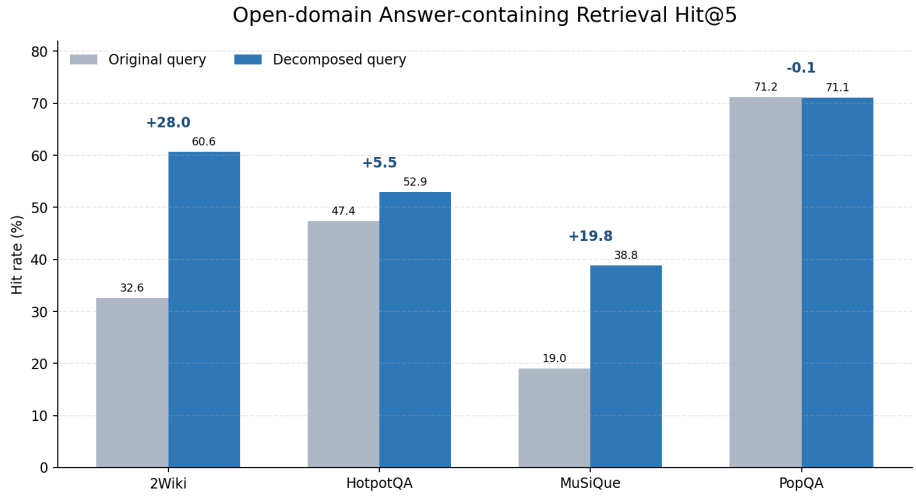


Fig. 5.1: Answer-containing recall under full-corpus retrieval. Decomposition improves multi-hop retrieval but has little effect on PopQA.

5.1.2 Gold and Base Decomposition on MuSiQue

To separate the value of decomposition from the value of the trained decomposer, we run a focused MuSiQue retrieval-recall analysis using the first 1,000 development questions. This slice contains only 2-hop questions, which makes the official MuSiQue gold decomposition directly comparable with SEEK's predicted two-step chain. Table 5.2 reports answer-containing Hit@5 under the same full-corpus Wikipedia and path-aware retrieval setting as Table 5.1.

SEEK substantially improves over original-question retrieval but remains below the oracle gold chain. The Gold raw condition falls back near the original-query result, showing that decomposition helps only when the bridge entity is resolved and substituted into the later query. Conditional analysis gives the same conclusion: SEEK approaches gold retrieval when the first-hop answer is correct and falls behind when it is wrong. The remaining gap is therefore driven primarily by bridge-answer drift rather than poor decomposition wording.

The untuned base model also clarifies why the decomposer is trained. Its query text

Table 5.2: MuSiQue answer-containing Hit@5 for original, SEEK, gold, and untuned-base decompositions.

Arm	Interpretation	Hit@5
Original question	No decomposition	19.0
SEEK	SEEK decomposition with predicted intermediate answer	38.8
Gold	Official MuSiQue decomposition with gold intermediate answer	**55.8**
Gold raw	Official decomposition without replacing `#1`	19.8
SEEK + gold bridge	SEEK sub-question text with gold intermediate answer	54.1
Untuned base + gold bridge	Llama-3-8B-Instruct without LoRA, using gold intermediate answer	52.9

can retrieve well under oracle bridge substitution, but it frequently violates the required step structure or output format. LoRA training therefore provides step-count control, valid placeholders, and compact BM25-friendly queries rather than merely improving wording.

5.2 Ablation Studies

This section tests whether retrieval remains necessary after decomposition, how the step-gate threshold affects retrieval and accuracy, and whether title handling should differ between the direct and decomposition paths.

5.2.1 Retrieval Is Necessary After Decomposition

Decomposition alone does not supply missing factual knowledge. Table 5.3 compares never retrieving, the headline accuracy-first gate, and always retrieving. For each policy, the stored no-retrieval and retrieved step answers are selected using Tail Leakage, and the resulting answer chain is resynthesised with the same adapter-disabled base-model prompt as the final system. Later resolved sub-questions are held fixed to the source trace, so this is a controlled fixed-chain comparison of retrieval policies rather than three independent end-to-end reruns.

Table 5.3: Step-level retrieval ablation after decomposition.

Dataset	Never retrieve	Confidence gate	Always retrieve	Gain (EM)	Gate retrieval rate
HotpotQA	29.80/39.38	37.80/**49.03**	**37.90/49.03**	+8.00	89.23%
2Wiki	33.90/39.86	**49.30/56.22**	49.30/56.15	+15.40	95.75%
MuSiQue	15.40/24.17	**26.50/34.63**	26.40/34.63	+11.10	94.78%
PopQA	24.80/25.93	**53.90**/55.73	**53.90/55.80**	+29.10	97.60%
Aggregate	25.98/32.34	**41.88/48.90**	**41.88/48.90**	+15.90	93.84%

Retrieval after decomposition is necessary: the accuracy-first gate clearly improves over never retrieving and remains effectively tied with always retrieving while skipping a small share of calls. The gate therefore controls retrieval cost, while the main accuracy gain comes from making evidence available after decomposition.

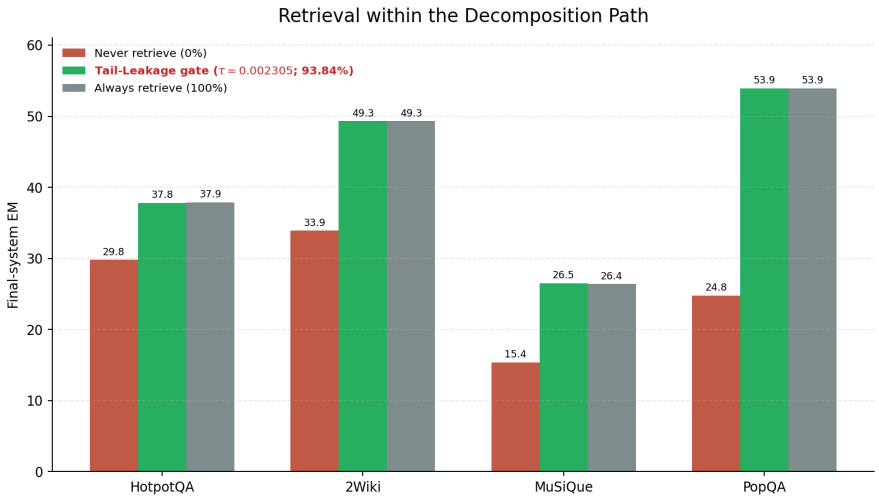


Fig. 5.2: Final-system EM across step-level retrieval policies.

At the accuracy-first threshold, SEEK skips 478 of 7,761 step-level retrieval calls across all four datasets (6.16%), affecting 418 of the 4,000 questions (10.45%). Table 5.4 compares SEEK with Always Retrieve on these affected questions. An omission is considered safe when it does not reduce question-level EM relative to Always Retrieve.

Only 16 affected questions (3.83%) lose EM correctness, indicating that the retrieval reduction is largely accuracy-neutral.

Table 5.4: Question-level outcomes among the 418 questions affected by step-level retrieval omission.

Safe omission: no EM loss	Harmful omission: EM loss
402 / 418 (96.17%)	**16 / 418 (3.83%)**

5.2.2 Gate Threshold Sensitivity

Table 5.5 replays the final system across step-level Tail Leakage thresholds. As in Table 5.3, later resolved sub-questions remain fixed and each selected step-answer chain is resynthesised before applying the final confidence verifier. The headline accuracy-first threshold is $\tau = 0.002305$. All rows are controlled fixed-chain sensitivity settings rather than independent end-to-end reruns.

Table 5.5: Final-system performance and retrieval rate across Tail Leakage thresholds.

τ	PopQA	HotpotQA	2Wiki	MuSiQue	Average	Retrieval rate
0.002305	**53.90/55.73**	**37.80/49.03**	**49.30/56.22**	26.50/34.63	**41.88/48.90**	93.84%
0.008461	53.70/55.53	37.50/48.72	48.70/55.63	**26.60/34.74**	41.62/48.66	87.82%
0.044170	53.70/55.53	36.40/47.33	47.50/54.57	24.80/33.45	40.60/47.72	75.51%
0.116500	51.90/53.74	36.30/47.22	45.80/52.83	23.70/32.69	39.42/46.62	64.40%

Figure 5.3 re-expresses the same sweep using the observed pooled retrieval rate on the horizontal axis. This view makes the operating-point trade-off easier to compare without introducing a second vertical scale.

The accuracy-first setting gives the strongest macro performance. A modest threshold increase reduces retrieval with little change in answer quality, whereas more aggressive thresholds lower both retrieval and accuracy. Most decomposed steps therefore still benefit from evidence under full-corpus Wikipedia search.

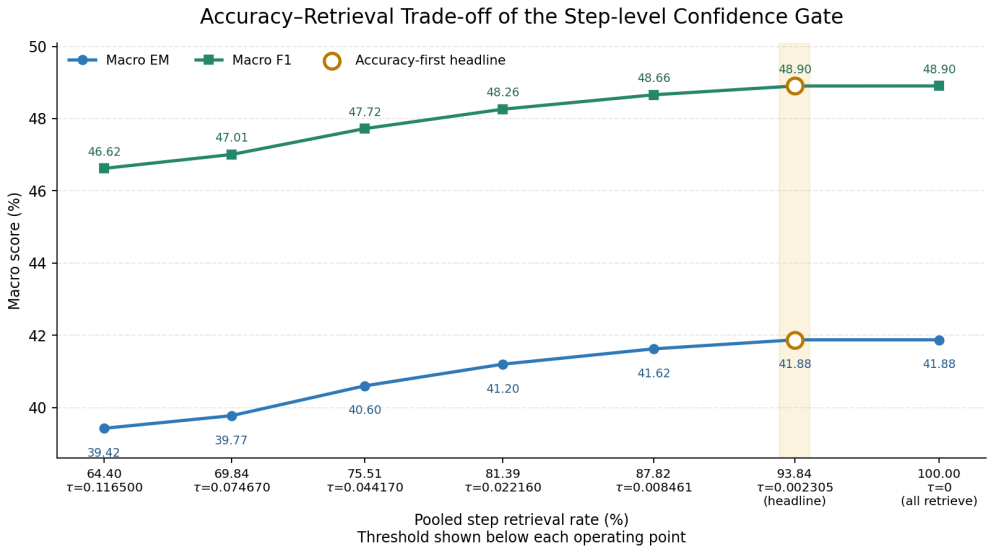


Fig. 5.3: Macro accuracy–retrieval trade-off of the step-level gate.

5.2.3 Path-Aware Title Handling

The original question and the decomposed sub-questions have different query forms. Original questions are longer and contain multiple constraints, whereas sub-questions are usually short and entity-centred. We therefore compare three BM25 title policies: title matching disabled for both paths, enabled for both paths, and the path-aware policy that disables title matching for the direct path but enables it for the decomposition path.

Table 5.6: End-to-end performance across title-retrieval policies (EM/F1).

Dataset	Title off	Title on	Path-aware
PopQA	51.9/54.0	52.4/54.6	**53.4/55.7**
HotpotQA	37.3/48.7	37.1/48.4	**37.9/49.3**
MuSiQue	23.3/31.6	24.9/**34.5**	**26.2**/34.4
2Wiki	44.5/50.4	49.1/55.5	**49.6/56.3**

Unlike the fixed-chain threshold replay, each title-policy condition in Table 5.6 is rerun end to end, so retrieval results, later contexts, and final answers may all change. The path-aware policy gives the best EM on every dataset and the strongest or nearly

strongest F1. Adaptive retrieval therefore concerns not only whether to retrieve, but also how retrieval should be configured for each query form.

5.3 The Two Answer Paths

The final system retains a direct answer candidate and a decomposition-path candidate. This section examines how the final confidence verifier divides work between them, where the candidates succeed separately, and how much routing headroom remains.

5.3.1 Verifier Routing Decisions

Table 5.7 reports the final verifier choice across all 1,000 questions in each dataset. The selection percentages describe routing behaviour, not answer correctness.

Table 5.7: Verifier selections by dataset.

Dataset	*N*	Direct chosen n (%)	Decomposition chosen n (%)
HotpotQA	1,000	171 (17.1%)	829 (82.9%)
2Wiki	1,000	25 (2.5%)	975 (97.5%)
MuSiQue	1,000	121 (12.1%)	879 (87.9%)
PopQA	1,000	10 (1.0%)	990 (99.0%)
Pooled	4,000	327 (8.2%)	3,673 (91.8%)

The verifier selects the decomposition candidate for most questions, while direct selection is most frequent on HotpotQA and MuSiQue. Candidate correctness is analysed next in Section 5.3.2.

PopQA produces a single-step structured reformulation rather than a multi-hop chain. This describes the form of the generated plan and is not, by itself, a retrieval- or answer-quality result.

5.3.2 Correctness Overlap Between Answer Paths

Table 5.8 compares the correctness of the two answer candidates before the final routing decision. Each of the 1,000 questions per dataset belongs to exactly one of four categories, separating path complementarity from routing quality.

Table 5.8: Correctness overlap between direct and decomposition candidates before final routing.

Dataset	Both paths correct	Only direct correct	Only decomposition correct	Neither path correct
HotpotQA	225	78	135	562
2Wiki	144	36	353	467
MuSiQue	75	27	192	706
PopQA	346	32	194	428

Decomposition contributes more unique correct answers on every dataset, as highlighted in Figure 5.4. The direct path remains complementary, especially on HotpotQA. A large share of the remaining multi-hop errors are wrong on both paths, so routing alone cannot remove them.

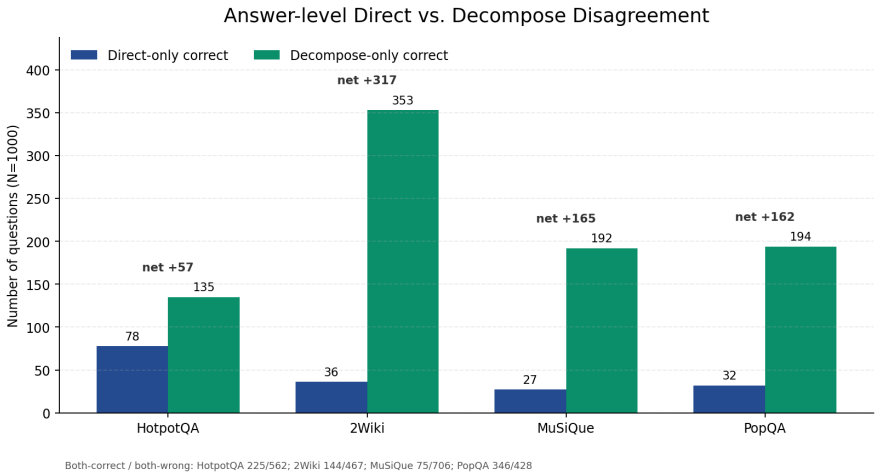


Fig. 5.4: Direct-only and decomposition-only correct answers by dataset.

At the accuracy-first setting, missing supporting evidence remains the largest source

of decomposition-path errors. The detailed taxonomy and supplementary diagnostic flags are reported in Appendix A.1.

5.3.3 Final Routing and Oracle Headroom

Because Table 5.8 shows that decomposition supplies most path-specific correct answers, always choosing the decomposition candidate is a strong routing baseline. Table 5.9 asks whether the final verifier improves on that baseline. Choosing the direct candidate recovers an answer when only the direct path is correct, but loses an answer when only the decomposition path is correct.

Table 5.9: Effect of the final verifier relative to always choosing decomposition.

Dataset	Direct-only cases recovered	Decomposition-only cases lost	Net correct answers vs. always decomposition
HotpotQA	39 / 78 (50.0%)	21 / 135 (15.6%)	**+18 (+1.80 EM)**
2Wiki	3 / 36 (8.3%)	7 / 353 (2.0%)	−4 (−0.40 EM)
MuSiQue	9 / 27 (33.3%)	11 / 192 (5.7%)	−2 (−0.20 EM)
PopQA	1 / 32 (3.1%)	2 / 194 (1.0%)	−1 (−0.10 EM)
Pooled counts / macro EM	52 / 173 (30.1%)	41 / 874 (4.7%)	**+11 (+0.28 EM)**

The verifier recovers some direct-only cases while preserving most decomposition-only wins, so it acts as a conservative fallback rather than the main source of SEEK's accuracy. Its small net benefit is concentrated on HotpotQA, while routing is slightly negative on the other datasets. The gold-label oracle in Figure 5.5 shows meaningful remaining path-selection headroom, although such an oracle is not deployable.

5.4 Tail Leakage as a Confidence Signal

To test whether Tail Leakage adds information beyond established confidence baselines, we group the evaluated signals into six conceptually distinct families and report one

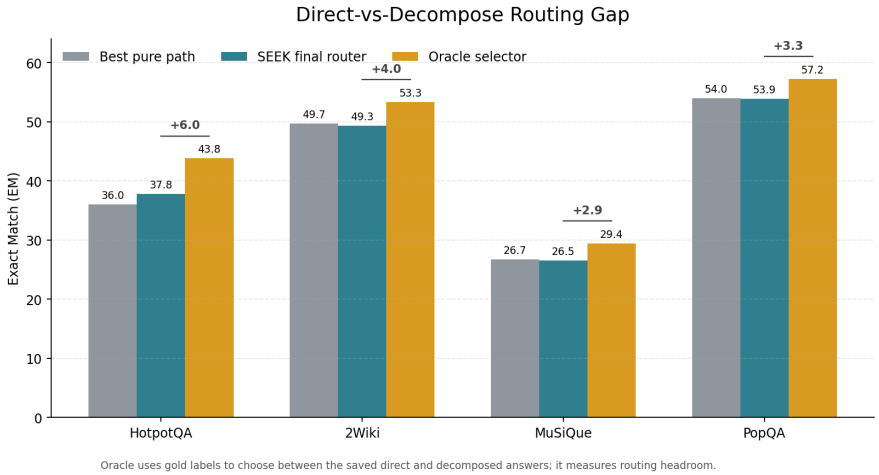


Fig. 5.5: Final routing performance and oracle headroom by dataset.

representative from each family. Maximum token entropy measures the overall spread of the next-token distribution, while sequence negative log-likelihood measures the probability assigned to the generated answer. The TARG margin uses the separation between the two leading logits [17]; sampled-answer disagreement measures response consistency, an established black-box confidence cue [26], [27]; and P(True) asks the model to assess its own proposed answer [22]. Tail Leakage instead measures how much probability escapes a small candidate set, as defined in Section 3.4. These families test different hypotheses about uncertainty rather than arbitrary transformations of one score.

Table 5.10 reports AUROC for predicting a wrong direct answer on the full trace of each dataset. All scores are oriented so that larger values mean greater uncertainty. Appendix A.2 gives concise definitions of the reported signals. Tail Leakage obtains the strongest macro result among the representatives. The comparison is diagnostic: small differences within these traces should not be interpreted as a universal ranking across models or datasets.

Table 5.11 reports the corresponding end-to-end comparison. Replacing maximum token entropy with Tail Leakage improves both macro EM and macro F1, although the gain is not uniform on every dataset.

Table 5.12 tests the mechanism proposed in Chapter 3. The amount of mass crossing from the leading candidates into the tail is the most informative entropy component, while

Table 5.10: AUROC of representative direct-path confidence signals.

Signal	HotpotQA	2Wiki	MuSiQue	PopQA	Macro
Tail Leakage	**0.757**	**0.717**	**0.726**	**0.798**	**0.749**
Maximum token entropy	0.743	0.671	0.711	0.780	0.726
Sequence negative log-likelihood	0.733	0.667	0.707	**0.798**	0.726
Sampled-answer disagreement	0.678	0.659	0.680	0.674	0.672
TARG logit-gap margin	0.669	0.589	0.618	0.682	0.639
P(True) self-evaluation	0.655	0.567	**0.726**	0.579	0.632

Table 5.11: End-to-end EM/F1 using maximum token entropy and Tail Leakage as the confidence signal.

Signal	HotpotQA	2Wiki	MuSiQue	PopQA	Macro
Maximum token entropy	35.50/46.11	**50.00/56.58**	24.70/33.06	51.90/54.71	40.52/47.61
Tail Leakage	**37.80/49.03**	49.30/56.22	**26.50/34.63**	**53.90/55.73**	**41.88/48.90**

the arrangement of probability within the tail is close to chance. Full entropy mixes these effects, whereas answer-level Tail Leakage retains the useful component and accumulates it across positions.

Table 5.12: Macro AUROC for direct-answer error detection when each entropy component is scored separately.

Signal	Macro AUROC
Top-five vs. tail	0.742
Within top five	0.712
Within tail	0.502
Token entropy	0.726
Answer-level Tail Leakage	**0.749**

The summed score also outperforms a weakest-token aggregation on longer answers, consistent with small leakage accumulating at several positions. A paired bootstrap confirms this advantage across all four datasets ($P < 0.001$ for the macro difference).

At the same time, Tail Leakage is not a correctness guarantee. Many wrong direct answers score as confidently as typical correct answers. Most are clean parametric hallucinations with essentially no leakage, while others are alias or answer-boundary near-misses. A confidence threshold cannot identify a confidently produced hallucination when the token distribution itself shows no warning. Appendix A.3 reports the detailed counts and error categories.

Table 5.13 shows the same limitation from the selective-answering side: questions are ranked by Tail Leakage, only the most confident fraction is kept, and accuracy is measured inside the kept set.

Table 5.13: Direct-path accuracy across confidence-coverage levels ranked by Tail Leakage.

Coverage	HotpotQA	2Wiki	MuSiQue	PopQA	Pooled
25%	57.6	34.0	22.4	79.2	52.8
50%	45.4	26.8	15.2	57.0	38.9
75%	38.0	22.1	12.5	46.3	30.2
100%	30.3	18.0	10.2	37.8	24.1

Accuracy rises steadily as coverage shrinks, showing that Tail Leakage provides a useful ranking. However, even the most confident subset remains unreliable on the harder multi-hop datasets, while single-hop PopQA is much stronger. The reverse failure also occurs: some correct answers receive high Tail Leakage. A high score alone is therefore a poor reason to discard an answer.

Both-wrong examples generally have higher Tail Leakage, but the overlapping distributions prevent reliable per-question error detection (Appendix A.3).

Chapter 6 Discussion

This chapter interprets the final results from Chapter 5. Taken together, they argue against reducing adaptive RAG to a single retrieve-or-skip decision. In multi-hop QA, the system must also decide when to trust decomposition, when to preserve the original question, and when the remaining error is outside the reach of routing.

6.1 Efficiency Considerations

SEEK does not guarantee fewer retrieval calls than other systems. Its main efficiency feature is simpler offline adaptation: only the decomposer is trained, while the retrieval gate and final verifier require no additional training. We therefore treat efficiency as a design consideration rather than a main empirical claim.

6.2 Why Direct and Decomposition Paths Must Coexist

The answer-level overlap analysis shows that decomposition contributes more unique correct answers on every dataset, while the direct path remains complementary. This complementarity is strongest on HotpotQA, where preserving the direct candidate matters most.

This is why SEEK keeps both paths. The direct path preserves the full question context, while the decomposition path serves as a retrieval strategy that exposes intermediate entities. Each path fails on different examples, and that is what makes the verifier necessary.

DeepRAG [10] generates each follow-up question from earlier answers. This supports adaptive planning but can also carry an incorrect intermediate answer into later questions. SEEK generates the full decomposition from the original question first. A wrong bridge answer may still hurt retrieval, but it cannot rewrite the remaining sub-questions, and the direct path remains available as a fallback.

The retrieval decision also works at the sub-question level. At the accuracy-first setting, SEEK retrieves for most multi-hop steps, although a chain may retrieve for only

some of its steps. Step-level retrieval rates and fully no-retrieval chains therefore measure different behaviours.

Original-question evidence preservation. Original-question evidence is retained in the decomposition path to preserve global context without changing the decomposer or training another retriever. Table 5.1 isolates query reformulation: it compares retrieval from the original question with retrieval from decomposed sub-questions and shows gains on the three multi-hop datasets. It does not test the evidence union, so this table cannot establish the union's independent contribution under the final configuration.

6.3 What Training the Decomposer Buys

The MuSiQue gold-decomposition analysis clarifies the role of decomposer training. SEEK improves retrieval over the original question, and its sub-question wording approaches gold performance when supplied with the correct bridge entity. The main bottleneck is therefore whether the first-hop bridge entity is answered correctly before the next query is formed.

The untuned base-model comparison gives the other half of the story. The base model can produce useful query text under oracle substitution but is not structurally reliable. LoRA training provides valid step counts, placeholders, and compact query style, making the decomposition path executable at inference time.

6.4 Tail Leakage: Useful Signal, Not Correctness Guarantee

Tail Leakage remains useful for deciding whether a sub-question should retrieve. The final retrieval-gate ablation compares never retrieving, the headline accuracy-first Tail Leakage threshold, and always retrieving, while the direct-path confidence battery shows that Tail Leakage has the strongest macro error-detection AUROC among the tested signals.

However, it is not a correctness guarantee. The direct-path and verifier-side analyses

both contain a stable confidently wrong region in which incorrect answers receive scores typical of correct answers. Appendix A.3 reports the dataset-level counts and error categories.

A compact way to state these findings is that confidence signals measure properties of the answer distribution, not the correctness of the reasoning behind it. The most common confident error is an incomplete reasoning path: the model stops at a bridge entity or returns an attribute instead of the requested comparison result, yet does so with the same near-zero leakage as a correct answer. A separate repeated-generation diagnostic reaches the same conclusion: many wrong answers remain identical across samples and have zero measured entropy, leaving disagreement-based detection with no variation to flag. Appendix A.3 gives the detailed breakdown. The reverse failure also deserves care because some correct answers appear uncertain, making a high Tail Leakage score an equally poor reason to discard them.

The final quadrant analysis reinforces this point: Tail Leakage separates both-wrong from both-correct examples on average, but the score distributions still overlap.

This explains why the final system uses confidence inside a broader verifier rather than as a single hard router. Tail Leakage locates uncertainty at the step level, but the direct-vs-decompose choice also needs answer type and decomposition stability. Even then, the remaining confidently wrong cases point toward future evidence-checking or self-consistency methods rather than another global threshold.

6.5 Remaining Headroom and Remaining Problems

The oracle analysis gives an upper bound for better routing and shows meaningful headroom over the current verifier on every dataset.

This headroom should be interpreted carefully. Oracle routing is not deployable because it uses the gold answer after the fact, and it cannot fix examples where both paths are wrong. Those cases require improvements below the router: better retrieval, reranking, bridge-answer propagation, or final synthesis.

The failure taxonomy in Table A.1 makes the bottleneck concrete: retrieval gaps dominate decomposition errors, followed by answering and synthesis failures, while constraint

loss is concentrated on HotpotQA. The remaining problem therefore splits into routing headroom, retrieval coverage, synthesis robustness, and confidently wrong answers.

6.6 Threats to Validity

External baseline numbers are paper numbers. The external baseline results in Table 4.2 are taken from the DeepRAG paper rather than reproduced in the exact same harness. Normalisation, prompting, and implementation details may differ slightly. To reduce protocol mismatch, SEEK follows DeepRAG's retrieval setup, and the reported averages are recomputed only over the four overlapping datasets.

Dataset slicing. All headline results use the first 1,000 dev questions. MuSiQue dev is sorted by hop depth, so this slice understates full-dev difficulty. Full-dev MuSiQue remains harder, especially for 4-hop questions.

Single seed and greedy decoding. All results use greedy decoding without seed averaging. Small differences should therefore be interpreted cautiously, while the larger PopQA and 2Wiki gains over DeepRAG are more robust.

Routing cannot fix both-wrong cases. The verifier can only choose between two existing candidates. If both direct and decomposed answers are wrong, routing cannot recover the example. This is the main reason oracle routing is an upper bound on routing quality rather than an upper bound on the whole QA system.

Chapter 7 Conclusions

7.1 Limitations

The principal limitations are:

- **HotpotQA remains below DeepRAG.** HotpotQA is the clearest case where direct and decomposed paths are complementary but the current verifier does not fully exploit their potential.
- **Verifier headroom remains.** Oracle direct-vs-decompose routing remains stronger than the current verifier, so path selection is not solved.
- **Confidence is not a correctness guarantee.** Tail Leakage is useful for sub-question retrieval gating, but some wrong answers look as confident as typical correct answers. The broader confidence-signal comparison likewise finds no decisive per-question error detector (Section 5.4).
- **Bridge-answer drift limits decomposition.** On MuSiQue, SEEK nearly matches gold decomposition when the first-hop bridge entity is correct, but wrong intermediate answers drag the second query away from the target evidence.
- **Both-wrong cases require better retrieval and synthesis.** Routing cannot fix examples where both direct and decomposed candidates are wrong, and retrieval gaps are the dominant decomposition-path failure on the multi-hop datasets.
- **Single-seed greedy decoding.** Variance estimates are not reported and would strengthen small-difference claims.

7.2 Future Work

Stronger verifier signals. The largest immediate opportunity is improving the verifier. Candidate signals include self-consistency disagreement, top-1 versus top-2 logit margin, hidden-state variance, answer-type compatibility, and lightweight learned calibration from

a small validation set. The signal battery in Chapter 5 suggests that gains from any single scalar signal are modest, so combining confidence with evidence-side checks is more promising than searching for another scalar. The goal is to approach the oracle direct-vs-decompose headroom without losing SEEK's interpretability.

Better retrieval and reranking. The large both-wrong counts show that routing alone is insufficient. Dense retrieval, hybrid BM25+dense retrieval, or reranking over the unioned evidence could improve cases where neither path retrieves enough supporting information.

Length-conditioned decomposer. A target-depth control token such as `[K=1]`, `[K=2]`, or `[K=4]` could reduce negative transfer between single-hop and multi-hop data. The verifier or a small complexity predictor could choose the target depth at inference time.

Multiple decompositions. Sampling multiple decompositions and selecting by final-answer confidence or agreement could reduce wrong-first-hop propagation. This is especially relevant because the MuSiQue analysis shows that bridge-answer drift, rather than decomposition wording alone, explains much of the gap between SEEK and gold decomposition. This would trade inference cost for robustness without requiring DPO.

Evidence-aware verification. Confidently wrong answers cannot be removed by another confidence threshold alone. Future verifiers should test whether the selected answer is externally supported by retrieved evidence, possibly with lightweight entailment checks, answer-type validation, or self-consistency over evidence snippets.

Span calibration. A small post-training or post-processing module for terse answer spans could close EM-only failures while preserving F1. This is especially relevant for verbose entity descriptions and comparison questions.

References

[1] L. Huang et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions", *ACM Transactions on Information Systems*, 2025.

[2] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey", *arXiv preprint arXiv:2312.10997*, 2023.

[3] R. Ren et al., "Investigating the factual knowledge boundary of large language models with retrieval augmentation", *arXiv preprint arXiv:2307.11019*, 2023.

[4] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions", in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[5] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity", in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.

[6] Y. Wang, P. Li, M. Sun, and Y. Liu, "Self-knowledge guided retrieval augmentation for large language models", in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

[7] Z. Jiang et al., "Active retrieval augmented generation", in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[8] W. Su, Y. Tang, Q. Ai, Z. Wu, and Y. Liu, "Dragin: Dynamic retrieval augmented generation based on the real-time information needs of large language models", in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[9] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-rag: Learning to retrieve, generate, and critique through self-reflection", in *International Conference on Learning Representations (ICLR)*, 2024.

[10] X. Guan et al., "Deeprag: Thinking to retrieve step by step for large language models", in *International Conference on Learning Representations (ICLR)*, 2026.

[11] S. Min, V. Zhong, L. Zettlemoyer, and H. Hajishirzi, "Multi-hop reading comprehension through question decomposition and rescoring", in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 6097–6109.

[12] T. Wolfson et al., "Break it down: A question understanding benchmark", *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 183–198, 2020.

[13] E. J. Hu et al., "Lora: Low-rank adaptation of large language models", in *International Conference on Learning Representations (ICLR)*, 2022.

[14] Y. Huang and J. Huang, "A survey on retrieval-augmented text generation for large language models", *arXiv preprint arXiv:2404.10981*, 2024.

[15] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, "When not to trust language models: Investigating effectiveness of parametric and non-parametric memories", in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[16] Z. Yao et al., "Seakr: Self-aware knowledge retrieval for adaptive retrieval augmented generation", in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[17] Y. Wang, Lu wei, and H. Ling, "Retrieval as a decision: Training-free adaptive gating for efficient rag", *arXiv preprint arXiv:2511.09803*, 2025.

[18] Su et al., "Fast or better? balancing accuracy and cost in retrieval-augmented generation with flexible user control", *CoRR*, 2025.

[19] T. Yu, S. Zhang, and Y. Feng, "Auto-rag: Autonomous retrieval-augmented generation for large language models", *arXiv preprint arXiv:2411.19443*, 2024.

[20] X. Li et al., "Search-o1: Agentic search-enhanced large reasoning models", *arXiv preprint arXiv:2501.05366*, 2025.

[21] W. Feng, C. Hao, Y. Zhang, J. Song, and H. Wang, "Airrag: Activating intrinsic reasoning for retrieval-augmented generation via tree-based search", *arXiv preprint arXiv:2501.10053*, 2025.

[22] S. Kadavath et al., "Language models (mostly) know what they know", *arXiv preprint arXiv:2207.05221*, 2022.

[23] Z. Yin, Q. Sun, Q. Guo, J. Wu, X. Qiu, and X. Huang, "Do large language models know what they don't know?", in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023.

[24] S. Kapoor et al., "Large language models must be taught to know what they don't know", *arXiv preprint arXiv:2406.08391*, 2024.

[25] X. Yin, X. Zhang, J. Ruan, and X. Wan, "Benchmarking knowledge boundary for large language models: A different perspective on model evaluation", *arXiv preprint arXiv:2402.11493*, 2024.

[26] Y. Yang, H. Yoo, and H. Lee, "Maqa: Evaluating uncertainty quantification in llms regarding data uncertainty", in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.

[27] L. Kuhn, Y. Gal, and S. Farquhar, "Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation", in *International Conference on Learning Representations (ICLR)*, 2023.

[28] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting hallucinations in large language models using semantic entropy", *Nature*, vol. 630, pp. 625–630, 2024.

[29] Z. Yang et al., "Hotpotqa: A dataset for diverse, explainable multi-hop question answering", in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

[30] X. Ho, A.-K. Duong Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps", in *Proceedings of the 28th International Conference on Computational Linguistics (COLING)*, 2020.

[31] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “Musique: Multihop questions via single-hop question composition”, *Transactions of the Association for Computational Linguistics (TACL)*, 2022.

[32] T. Kwiatkowski et al., “Natural questions: A benchmark for question answering research”, *Transactions of the Association for Computational Linguistics (TACL)*, 2019.

[33] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, “Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension”, in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017.

[34] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “Squad: 100,000+ questions for machine comprehension of text”, in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2016.

[35] A. Rényi, “On measures of entropy and information”, in *Proceedings of the Fourth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Contributions to the Theory of Statistics*, University of California Press, 1961, pp. 547–561.

[36] V. Karpukhin et al., “Dense passage retrieval for open-domain question answering”, in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550

[37] Y. Zheng et al., “Llamafactory: Unified efficient fine-tuning of 100+ language models”, in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, 2024.

Appendices

Appendix A Supplementary Material

This appendix provides supporting analyses for Chapters 5 and 6: decomposition failures, confidence-signal definitions, and confidence errors.

A.1 Decomposition Failures

We use the final accuracy-first step gate ($\tau = 0.002305$) and assign one main cause to each incorrect decomposition answer. Step answers are reselected and resynthesised at this threshold, while the sub-question wording stays fixed. Evidence is covered when the required Wikipedia titles appear in the top 10 results from either the original question or the resolved sub-questions.

Table A.1: Main causes of decomposition errors.

Primary cause	HotpotQA ($N = 640$)	2Wiki ($N = 503$)	MuSiQue ($N = 733$)
Missing required evidence	307 (48.0%)	311 (61.8%)	513 (70.0%)
Incorrect step answer with complete evidence	131 (20.5%)	98 (19.5%)	73 (10.0%)
Correct steps but wrong final answer	164 (25.6%)	92 (18.3%)	138 (18.8%)
Lost constraint; original retrieval has complete evidence	38 (5.9%)	2 (0.4%)	9 (1.2%)

Missing evidence is the largest failure category on all three multi-hop datasets. Other errors mainly come from incorrect step answers or final synthesis. Lost constraints are less common and occur mostly on HotpotQA.

Table A.2 shows additional patterns that may overlap. The last three rows separately show how often retrieval was used.

The first two rows describe plan or synthesis problems and may overlap. The last three rows show retrieval use, not whether the retrieved evidence was sufficient.

Table A.2: Additional patterns in decomposition errors.

Diagnostic flag	HotpotQA (N = 640)	2Wiki (N = 503)	MuSiQue (N = 733)
Repeats an intermediate answer	142 (22.2%)	22 (4.4%)	68 (9.3%)
First sub-question repeats the original question	118 (18.4%)	154 (30.6%)	235 (32.1%)
No steps retrieve	7 (1.1%)	4 (0.8%)	0 (0.0%)
Some steps retrieve	93 (14.5%)	21 (4.2%)	54 (7.4%)
All steps retrieve	540 (84.4%)	478 (95.0%)	679 (92.6%)

A.2 Confidence Signals

Table A.3 defines the confidence signals used in Section 5.4. Here, $p_t(v)$ is a next-token probability, y_t is the generated token, and T is the answer length. A higher score means greater uncertainty.

Table A.3: Confidence signal definitions.

Signal	Definition
Maximum token entropy	Largest token-level entropy in the answer: $\max_t[-\sum_v p_t(v) \log p_t(v)]$ [17], [26].
Answer likelihood	Total negative log-likelihood of the generated answer: $-\sum_{t=1}^{T} \log p_t(y_t)$ [27].
TARG margin	Average transformed gap between the two largest logits over the first eight tokens [17].
Sample disagreement	Fraction of ten sampled answers that differ from the greedy answer [26], [27], [28].
P(True) self-check	Model-estimated probability that its answer is false: $1 - P(\text{True} \mid q, C, \hat{a})$ [22].
Tail Leakage	Cumulative probability leakage outside the top-five token set: $-\sum_{t=1}^{T} \log \sum_{v \in V_5(p_t)} p_t(v)$.

All signals use the same direct-path outputs and are ranked by AUROC for exact-match errors. Except for sample disagreement and P(True), they come from the same greedy decoding pass.

A.3 Confidence Errors

Tail Leakage is useful overall but cannot guarantee that one answer is correct. Table A.4 compares cases where both paths are correct with cases where both are wrong. Figure A.1 shows the score distributions.

Table A.4: Tail Leakage by correctness.

Dataset	Mean, both correct	Mean, both wrong	AUROC	Confidently wrong
HotpotQA	0.01260	0.12927	0.777	15.5%
2Wiki	0.03414	0.20215	0.751	20.1%
MuSiQue	0.04945	0.25025	0.766	12.7%
PopQA	0.00974	0.10282	0.846	4.9%
Pooled	0.01877	0.17926	0.817	9.3%

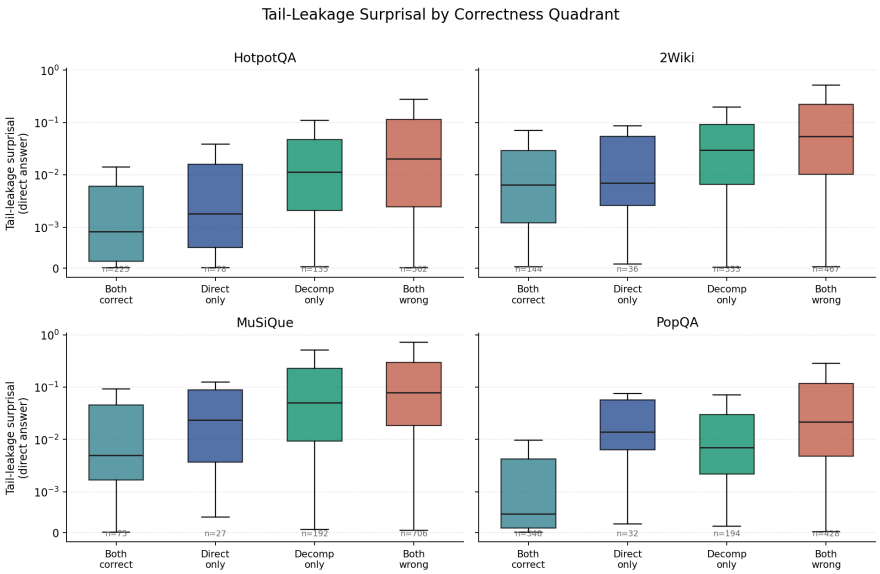


Fig. A.1: Tail Leakage by correctness group.

Both-wrong cases usually have higher Tail Leakage, but the distributions overlap. We call a wrong answer confidently wrong when its score is no higher than the median score of correct answers in the same dataset. Table A.5 shows the error types.

Table A.5: Confidently wrong answers by error type.

Category	HotpotQA	2Wiki	MuSiQue	PopQA	Pooled
Wrong direct answers	697	820	898	622	3,037
Wrong entity	55	116	89	21	281
Boundary or alias mismatch	52	43	31	22	148
Partial name overlap	9	22	9	0	40
Wrong date or number	5	1	16	0	22
All confidently wrong	**121 (17%)**	**182 (22%)**	**145 (16%)**	**43 (7%)**	**491 (16.2%)**

We also tested repeated sampling with a fixed BM25 context. This test is separate from the final routing policy and still uses retrieved evidence. Table A.6 counts cases with 15 identical answers and zero mean maximum token entropy. An answer is wrong if it fails both normalised exact match and token F1.

Table A.6: Repeated-sampling agreement and errors.

Dataset	Identical answers	Identical and zero entropy	Wrong cases	Wrong rate
HotpotQA	620	522	127	24.3%
2Wiki	368	301	169	56.1%
MuSiQue	403	355	188	53.0%
PopQA	478	360	136	37.8%
Pooled	1,869	1,538	620	40.3%

When all samples agree, disagreement cannot flag these errors.